



## **HomeGigs Application**

Ali Abdullah (200518299), Chiemego Emmanuel Nwokolo (200513473),  
Chinedu Godswill Nwoye (200521504), Nyabijek Gatdet (200479520)

Department of Engineering, University of Regina

ENSE 281: Software Engineering Management (Winter - 2026)

Professor Dr. Tim Maciag

April 08, 2026

# INTRODUCTION

On one hand, a disturbing number of skilled workers in the job market struggle with autonomy, forced company commitments, limited available variety of work, slow career progression, reduced income control, and much more. On the other hand, homeowners and small business owners are tired of paying massive amounts to different firms or big players in this business, for even small daily routine gigs. It won't be wrong to state that the job market has become controlled and monopolized. There is a huge gap between both ends that needs to be studied, reflected upon, and bridged together to enhance the economy and provide better financial decision options for both sides. This problem gave birth to our team project: HomeGigs.

HomeGigs was created to tackle these problems by allowing the homeowners or small business owners to post gigs regarding the work they wanted, and by providing autonomy to skilled workers by letting them choose amongst gigs posted and allowing them to explore other skills they can render on this platform too. This motive was well accomplished by creating a web-based software application that serves as a bridge between skilled workers and jobs, taking the middlemen out of the equation and leaving it just between the workers and clients. As the name implies, HomeGigs basically aims for common household jobs, so the target customers do not have to pay huge amounts for small daily routine jobs, and the workers can get paid a better amount as well.

Upon finalization of the concept of the application, the team allocated two and a half months to the project. The timeframe included research to understand the job market, average homeowners' problems, average skilled workers' earnings, and learning front-end and back-end technologies to create the web-based software application. The time span was divided into four different categories as project pitch, project prerequisites and designs, project check-in, and project delivery and closing.

# PROJECT GOALS

## **PRIMARY GOAL**

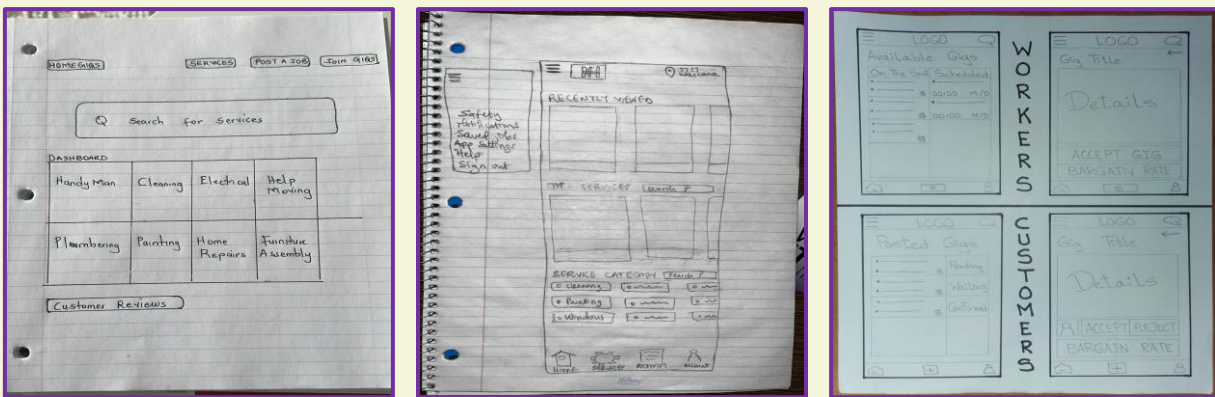
The primary goal and minimum viable product of the web-based software application for this project is to let a customer post a gig and a worker accept the gig.

## **SECONDARY GOALS**

1. **Creating a dual portal system:** This was aimed at having two unique interfaces - one for the customer and one for the worker, with their varying functionality that allows them to perform their specific tasks and achieve specific goals.
2. **Creating a Gig marketplace:** This was aimed at providing a gig marketplace where homeowners can post jobs that they need to get done with reasonably necessary details like job title, description, home address, price and images. The workers, on the other hand, would have access to these after they have been posted.
3. **Creating a chat system for bargaining:** This was aimed at providing a secure and monitored chat system for the customer and worker to agree on a suitable price, to produce a trustworthy web application and prevent fraud.
4. **Creating a verification process for users:** This was aimed at preventing fraud and ensuring that real people are creating accounts on the website. It would involve scanning a user's face and comparing it to an identity card or any alternative. This goal was not achieved as it involves sophisticated programming knowledge that the team is looking to explore in the future.

# PROJECT EXPLORATION AND IDEATION

Initially, the team prepared a number of low-fidelity prototypes (with pen and paper) that had both users, workers and homeowners, using the same interface. Upon further considerations, the team realized that both users have different role-specific functionality; customers post gigs and track their status, while the workers browse gigs, accept them, and view earnings. This established that creating a single portal system would create clutter and confusion.



Upon finalizing this decision, the team created a high-fidelity prototype (on Figma) to dive deeper into what was being created and ensure that all team members were on the same page. Working on this high-fidelity prototype as a team provided an ease of idea generation, and the team could clearly see what would work and what would not. It also helped visualize the customers' and workers' user interface, gig marketplace, chat system, gig status updates, and other functionalities.



# SYSTEM ARCHITECTURE

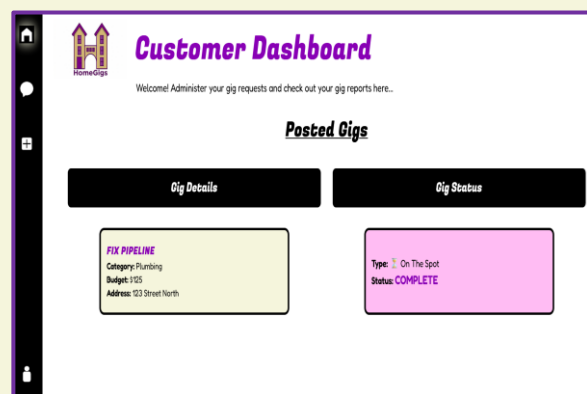
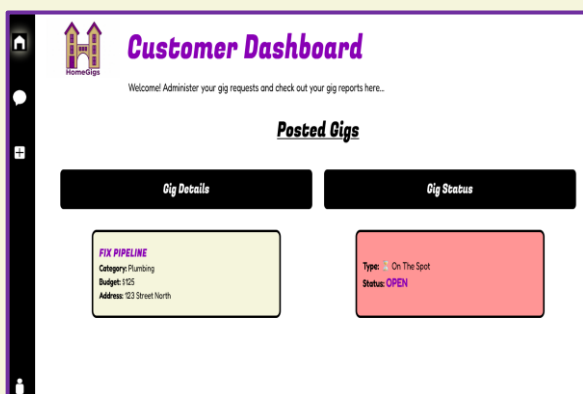
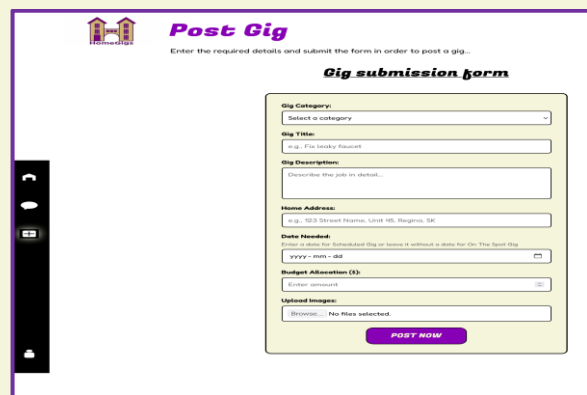
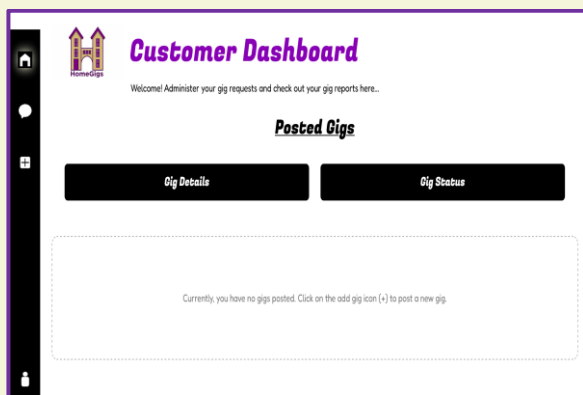
The team implemented the Model View Controller (MVC) architecture for the programming stage of our web-based software application.

1. **Model:** The team implemented MongoDB, an open-source NoSQL document-oriented database, to store gig data, earnings record, and user information. As the model in this architecture, MongoDB was responsible for storing this important information and passing it to the controller architecture or to the view architecture when the user requests that information.
2. **View:** The team used HTML files to highlight the structure of the website. This is what the user sees and uses to send information to the back-end. Separate files were created for every page that the user could have access to; this includes the customers' and workers' dashboard, earnings page, login page, register page, post gigs page, chats page, and more. These pages were eventually converted to EJS files for proper communication with the backend - the controller architecture. During this whole procedure, one CSS file was used to implement and store different classes associated with the designing features of the application.
3. **Controller:** The team worked upon JavaScript files and logics that were responsible for mediating between the users' requests, the model architecture, and the view architecture. This part of the project receives every request that the user inputs through the web-based application and performs the necessary actions.

# KEY USER INTERACTIONS

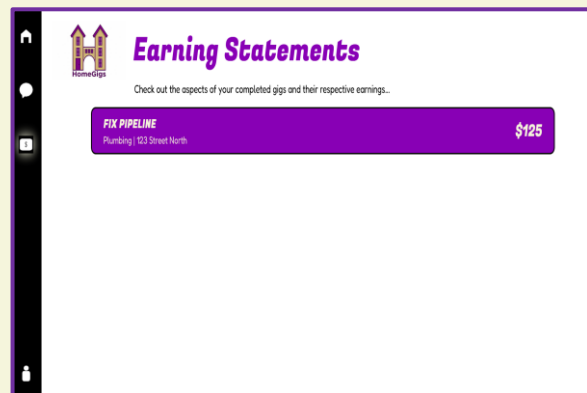
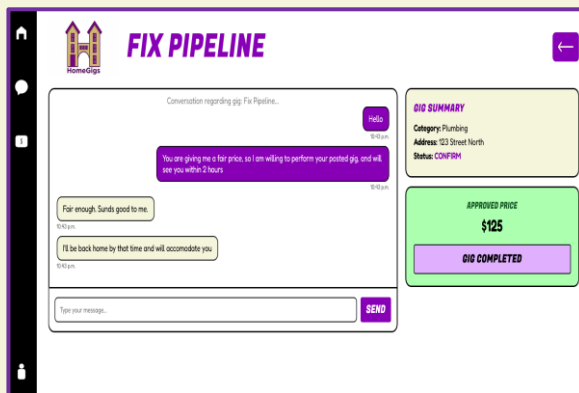
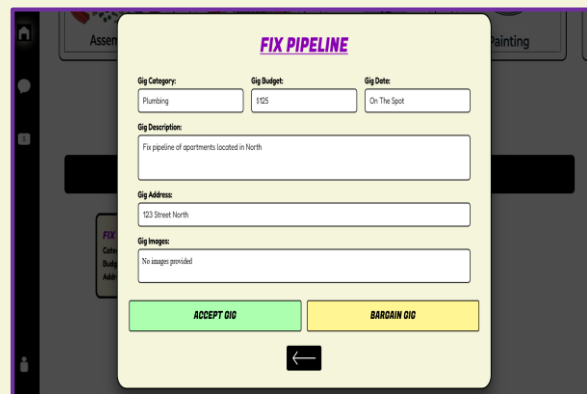
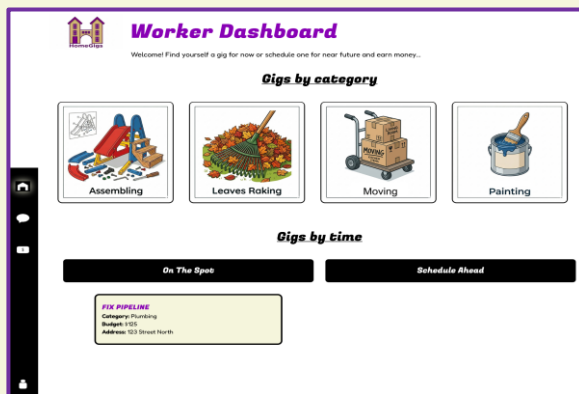
## CUSTOMER FLOW

A flow example for a customer would go as follows: the customer submits the post-gig form with the details of the gig, the Controller receives this post request, validates it and saves it in the Model (MongoDB), and the customer is redirected to View, the customer's dashboard page, to see the posted gig. The provided images can help understand the customer flow in a better way:



# WORKER FLOW

A flow example for a worker would go as follows: the worker visits the worker's dashboard page and clicks on the gig tile, the Controller fetches that gig from Model (MongoDB), the data is passed to the View, and the worker is presented by a pop-up to see relevant actions for that gig. The provided images can help understand the worker flow in a better way:



# FEEDBACK AND ITERATIONS

The team implemented a scrum system where the members engaged in usability testing with usability questionnaires for every sprint. Some of the feedback comments that had a reasonable impact on the design are following:

1. **Proximity of the accept and reject buttons:** When providing a demo of the high fidelity prototype in the usability testing, user feedback was received stating that the reject and accept gig button in the chat system were too close and could lead to a user picking the wrong option by mistake. To fix this, the team removed them from chat and implemented a new popup mechanism with some modifications.
2. **Abandon button that comes after accepting gig:** Originally, after the user clicks on the accept gig button, they are redirected to a page where they get to pick between the “abandon” and “finished” buttons. It was made clear to the team that this was unnecessary as the user has already clicked on the option to accept the gig. To fix this, the team completely removed the concept of these buttons while designing.
3. **Allocation of responsibilities in a situation of misunderstanding/unsatisfactory work:** A user wondered who would be responsible for unsatisfactory work or conflict between the customer and the worker. To handle this, the team decided to set up a team of support members who would be trained to provide solutions.
4. **Safety conditions at the workplace:** A user expressed concerns about the safety of the workers when working and who would be liable for whatever happens. To handle this, the team decided to reflect upon this idea with some insurance companies to see if there is a way to utilize liability insurance for certain incidents.

# DEEPER REFLECTIONS

Once the team completed the project and introduced others to the new engaging platform named HomeGigs, the team went into having conversations with others around, and had a moment to self-reflect upon things and find some answers to hidden questions, that could be concluded as:

1. **Overcoming obstacles:** The team decided to appreciate every member's ideas and concerns regarding the project, to avoid conflicts, but sometimes, the team members had to be brutally honest with each other to develop a better product. The team members had mutual respect that helped them glue together and progress.
2. **Team Feelings:** The team felt that despite having different thoughts and opinions on several things, all members can settle together on one page by following the ideology behind False-Consensus theorem, and not assuming themselves as the only users that will be utilizing this application ahead.
3. **Proud Moment:** The team was very proud when it found out that all MVPs were accomplished, and all functionality was achieved after putting in tons of effort. The team was proud that it not only brought functionality to life but also had such an attractive and funky sort of visual layout that would develop interest of the users.
4. **Future Strategy:** The team had numerous ideas about bringing this product from just an application to be utilized on local systems, to the open wide world. The team's motive was not only to develop an application to bridge the gap between both ends and see if it works well, but the team really meant every effort that was put into the whole procedure within the time span of two and a half months.
5. **Lessons Learned:** The team learned several lessons throughout their journey of efforts, but the mentionable ones are not taking time for granted, staying in constant communication, putting in genuine hard work and efforts, but not stressing a lot. **Do your best! Leave the rest!**